C programming has three types of loops:

1. for loop
2. while loop
3. do...while loo

for Loop

The syntax of the for loop is:

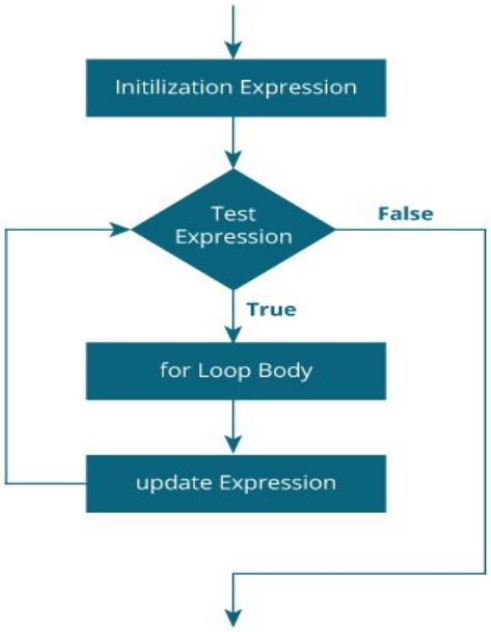
```
for (initializationStatement; testExpression; updateStatement)
{
   // statements inside the body of loop
}
```

How for loop works?

- The initialization statement is executed only once.
- Then, the test expression is evaluated. If the test expression is evaluated to false, the for loop is terminated.
- However, if the test expression is evaluated to true, statements inside the body of the for loop are executed, and the update expression is updated.
- Again the test expression is evaluated.

This process goes on until the test expression is false. When the test expression is false, the loop terminates.

for loop Flowchart



Example 1: for loop

```
// Print numbers from 1 to 10
#include <stdio.h>
int main() {
  int i;
  for (i = 1; i < 11; ++i)
  {
    printf("%d ", i);
  }
  return 0;
}
```

Output

1 2 3 4 5 6 7 8 9 10

1. i is initialized to 1.
2. The test expression i < 11 is evaluated. Since 1 less than 11 is true, the body of for loop is executed. This will print the **1** (value of i) on the screen.
3. The update statement ++i is executed. Now, the value of i will be 2. Again, the test expression is evaluated to true, and the body of for loop is executed. This will print **2** (value of i) on the screen.
4. Again, the update statement ++i is executed and the test expression i < 11 is evaluated. This process goes on until i becomes 11.
5. When i becomes 11, i < 11 will be false, and the for loop terminates.

while loop

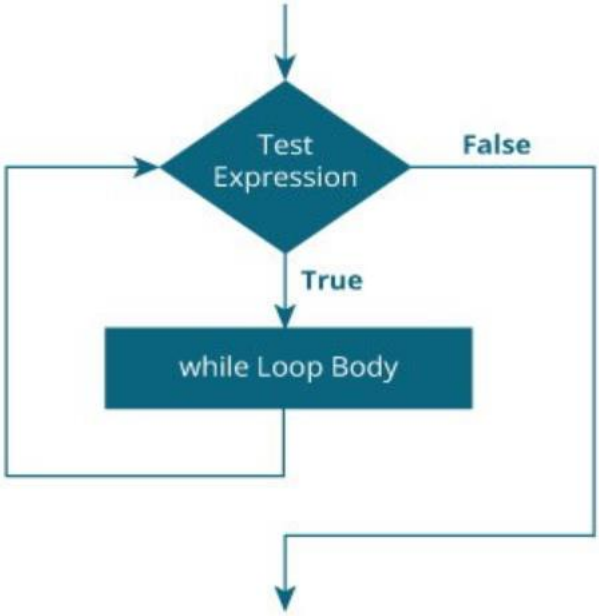
The syntax of the while loop is:

```
while (testExpression) {
  // the body of the loop
}
```

How while loop works?

- The while loop evaluates the testExpression inside the parentheses ().
- If testExpression is **true**, statements inside the body of while loop are executed. Then, testExpression is evaluated again.
- The process goes on until testExpression is evaluated to **false**.
- If testExpression is **false**, the loop terminates (ends).

Flowchart of while loop



Example: while loop

```
// Print numbers from 1 to 5
#include <stdio.h>
int main() {
  int i = 1;
  while (i <= 5) {
    printf("%d\n", i);
    ++i;
  }
  return 0;
}
```

Output

```
1
2
3
4
5
```

i). Here, we have initialized i to 1.

ii). When i = 1, the test expression i <= 5 is **true**. Hence, the body of the while loop is executed. This prints 1 on the screen and the value of i is increased to 2.

iii). Now, i = 2, the test expression i <= 5 is again **true**. The body of the while loop is executed again. This prints 2 on the screen and the value of i is increased to 3.

iv). This process goes on until i becomes 6. Then, the test expression i <= 5 will be **false** and the loop terminates.

do...while loop

The do..while loop is similar to the while loop with one important difference. The body of do...while loop is executed at least once. Only then, the test expression is evaluated.

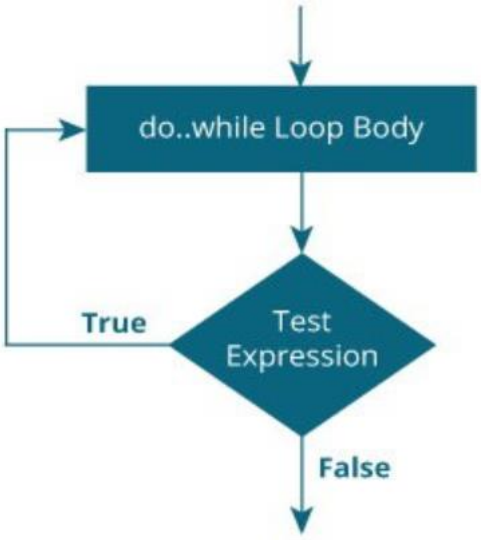
The syntax of the do...while loop is:

```
do {
  // the body of the loop
}
while (testExpression);
```

How do...while loop works?

- The body of do...while loop is executed once. Only then, the testExpression is evaluated.
- If testExpression is **true**, the body of the loop is executed again and testExpression is evaluated once more.
- This process goes on until testExpression becomes **false**.
- If testExpression is **false**, the loop ends.

Flowchart of do...while Loop



Example: do...while loop

```
// Program to add numbers until the user enters zero
#include <stdio.h>
int main() {
  double number, sum = 0;
  // the body of the loop is executed at least once
```

```
  do {
    printf("Enter a number: ");
    scanf("%lf", &number);
    sum += number;
  }
  while(number != 0.0);
  printf("Sum = %.2lf",sum);
  return 0;
}
```

Output

```
Enter a number: 1.5
Enter a number: 2.4
Enter a number: -3.4
Enter a number: 4.2
Enter a number: 0
Sum = 4.70
```

The do...while loop executes at least once i.e. the first iteration runs without checking the condition. The condition is checked only after the first iteration has been executed.

```
do {
  printf("Enter a number: ");
  scanf("%lf", &number);
  sum += number;
}
while(number != 0.0);
```

So, if the first input is a non-zero number, that number is added to the sum variable and the loop continues to the next iteration. This process is repeated until the user enters 0.

But if the first input is 0, there will be no second iteration of the loop and sum becomes 0.0.

Outside the loop, we print the value of sum.

Functions in C

A **function** is a block of statements that performs a specific task. Functions are used to divide a large program into smaller and manageable parts.

Definition of Function

A function in C consists of:

- **Return type**
- **Function name**
- **Parameters/Arguments**

- **Function body**

Syntax

```
return_type function_name(parameter_list)
{
   // statements
   return value;
}
```

Example

```
int add(int a, int b)
{
   int sum;
   sum = a + b;
   return sum;
}
```

Here:

- int → return type
- add → function name
- a, b → parameters
- return sum → returns the result

Function Call

A **function call** is used to execute a function.

Syntax

```
function_name(arguments);
```

Example

```
int result;
result = add(10, 20);
```

Here, add(10, 20) is the function call.

Return Statement

The **return statement** is used to:

1. Return a value from a function to the calling function.
2. Terminate the execution of the function.

Syntax

return expression;

Example

```
int square(int n)
{
   return n * n;
}
```

The value of n * n is returned to the calling function.

For a function that does not return a value:

```
void display()
{
   printf("Hello");
   return;
}
```

Complete Example of Function

```
#include <stdio.h>
int add(int a, int b)
{
   return a + b;
}
int main()
{
   int result;
   result = add(10, 20);
   printf("Sum = %d", result);
   return 0;
}
```

Output:

Sum = 30

Call by Value and Call by Reference in C

1. Call by Value

In **call by value**, a copy of the actual argument is passed to the function. Any change made to the formal parameter inside the function **does not change the original variable**.

Syntax

function_name(variable);

Program

```
#include <stdio.h>

void change(int x)

{

x = 100;

}

int main()

{

int a = 10;

printf("Before function call: a = %d\n", a);

change(a);

printf("After function call: a = %d\n", a);

return 0;

}
```

Output

Before function call: a = 10

After function call: a = 10

Detailed Explanation

Step 1: Variable declaration

int a = 10;

A variable a is created and its value is 10.

a = 10

Step 2: Function call

change(a);

The value of a is passed to the function.

A **copy** of 10 is given to x.

a = 10 ————> x = 10

(original) (copy)

Step 3: Function execution

void change(int x)

{

x = 100;

}

The value of x is changed from 10 to 100.

x = 10

↓

x = 100

But x is only a copy of a. Therefore, a is not changed.

a = 10

x = 100

Step 4: After function call

When the function finishes, x is destroyed.

The original variable still contains:

a = 10

Therefore, the output is:

After function call: a = 10

2. Call by Reference

In **call by reference**, the address of the actual argument is passed to the function. The function can use this address to access and modify the original variable.

In C, call by reference is implemented using **pointers**.

Syntax

function_name(&variable);

Program

```
#include <stdio.h>
void change(int *x)
{
*x = 100;
}
int main()
{
int a = 10;
printf("Before function call: a = %d\n", a);
change(&a);
printf("After function call: a = %d\n", a);
return 0;
}
```

Output

Before function call: a = 10

After function call: a = 100

Detailed Explanation

Step 1: Variable declaration

int a = 10;

A variable a is created with value 10.

Suppose its memory address is 1000.

Variable	Value	Address
a	10	1000

Step 2: Function call

change(&a);

The & operator gives the **address of a**.

Therefore:

&a = address of a

The address is passed to the function.

a

10

│

│ (address)

↓

1000

Step 3: Function parameter

void change(int *x)

Here, x is a **pointer variable**.

It stores the address of a.

x = 1000

Therefore, x points to a.

x

│

│ points to

↓

a = 10

Step 4: Dereferencing

*x = 100;

The * operator is used to access the value stored at the address contained in x.

Since x contains the address of a:

*x

↓

a

Therefore:

*x = 100;

changes the value of a to 100.

Before:

a = 10

After:

a = 100

Step 5: After function call

When the function finishes, the original variable a has already been changed.

Therefore:

a = 100

Meaning of & and *

& Address-of Operator

The & operator gives the memory address of a variable.

&a

means **address of a**.

*** Dereference Operator**

The * operator is used with a pointer to access the value stored at the address.

*x

means **value stored at the address contained in x**.

Difference Between Call by Value and Call by Reference

Call by Value	**Call by Reference**
Copy of value is passed.	Address of variable is passed.
Original variable is not changed.	Original variable can be changed.
Pointer is not required.	Pointer is required in C.
Function call: change(a)	Function call: change(&a)
Parameter: int x	Parameter: int *x
Modification is made to the copy.	Modification is made to the original variable.

Call by Value

a = 10

↓

change(a)

↓

x = 10

↓

x = 100

a remains 10

Call by Reference

a = 10

↓

change(&a)

↓

x stores address of a

↓

*x = 100

↓

a becomes 100

Recursion in C

Recursion is a process in which a function calls itself repeatedly until a specified condition is satisfied.

A recursive function contains two important parts:

- **Base condition:** Stops the recursion.
- **Recursive call:** Calls the same function again.

2. Factorial Using Recursion

The factorial of a number is the product of all positive integers from 1 to that number.

$$n! = n \times (n-1)!$$

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Program

```
#include <stdio.h>
int factorial(int n)
{
   if (n == 0 || n == 1)
      return 1;
   return n * factorial(n - 1);
}
int main()
{
   printf("Factorial = %d", factorial(5));
   return 0;
}
```

Output:

Factorial = 120

Explanation

- factorial(5) returns 5 * factorial(4).
- factorial(4) returns 4 * factorial(3).
- This continues until factorial(1) returns 1.
- The results are multiplied: 5 × 4 × 3 × 2 × 1 = 120.

Fibonacci Series Using Recursion

The Fibonacci series is a sequence in which each number is the sum of the previous two numbers.

Example: 0, 1, 1, 2, 3, 5, 8, 13, ...

Program

```
#include <stdio.h>
int fibonacci(int n)
{
   if (n == 0)
```

```
    return 0;
  if (n == 1)
    return 1;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
int main()
{
  int i;
  for (i = 0; i < 7; i++)
  {
    printf("%d ", fibonacci(i));
  }
  return 0;
}
```

Output:

0 1 1 2 3 5 8

Explanation

- fibonacci(0) returns 0.
- fibonacci(1) returns 1.
- For other numbers, the function returns fibonacci(n-1) + fibonacci(n-2).
- The for loop prints the first seven Fibonacci numbers.